

CODI AI · BACKEND API v1

프론트엔드 연동 가이드

보유 의류, TPO, 날씨, 신체 프로필을 기반으로 코드를 추천하고 2D/AI
착장 이미지를 생성하는 Spring Boot REST API의 실전 연동 문서

대상 프론트엔드 개발자

기준 소스 Codi AI Backend 2.0.0

문서 기준일 2026-08-16

엔드포인트 34개 · Base path `/api/v1`

0. 문서 사용법

- | | |
|-----------------------|----------------------|
| 1. API 기본 규칙과 인증 | 7. TPO·날씨 API |
| 2. 프론트 권장 연동 순서 | 8. 코디 추천 API |
| 3. 전체 엔드포인트 요약 | 9. 가상 착장 API |
| 4. 인증 API | 10. 찜·피드백 API |
| 5. 사용자·신체 프로필·아바타 API | 11. 공통 DTO 필드 사전 |
| 6. 옷장 API | 12. TypeScript 연동 예시 |
| | 13. 오류 처리와 QA 체크리스트 |

실행 중 자동 문서

Swagger UI: `{BASE_URL}/swagger-ui.html`

OpenAPI JSON: `{BASE_URL}/v3/api-docs`

이 PDF는 현재 컨트롤러, DTO validation, 서비스 로직을 기준으로 작성했다. 문서의 `{BASE_URL}` 은 로컬에서 `http://localhost:8080` , Render 배포 후에는 실제 `https://....onrender.com` 주소로 바꾼다.

1. API 기본 규칙과 인증

1.1 기본 주소와 Content-Type

구분	규칙
API Base	{BASE_URL}/api/v1
JSON 요청	Content-Type: application/json
이미지 업로드	multipart/form-data . 브라우저가 boundary를 설정하도록 Content-Type을 직접 넣지 말고 FormData 를 전달한다.
일시	ISO 8601. 예: 2026-08-16T19:00:00+09:00
ID	UUID 문자열. 프론트에서 숫자로 변환하지 않는다.

1.2 JWT 인증

/api/v1/auth/** 만 인증 없이 호출할 수 있다. 나머지 API는 모두 아래 헤더가 필요하다.

```
Authorization: Bearer {accessToken}
```

401을 받으면 access token이 없거나 만료된 것이다. 저장한 refresh token으로 `POST /auth/refresh` 를 호출한 뒤 원래 요청을 1 회만 재시도한다. 마음대로 무한 재시도하지 않는다.

1.3 공통 성공 응답

데이터를 반환하는 API는 대부분 다음 envelope를 사용한다.

```
{
  "data": { ... },
  "meta": {
    "requestId": "req..."
  }
}
```

`DELETE` , logout, 추천 피드백처럼 응답이 `204 No Content` 또는 body 없는 `201` 인 API도 있다. 이때 JSON parsing을 시도하지 않는다.

1.4 공통 오류 응답

```
{
  "code": "VALIDATION_ERROR",
  "message": "요청값을 확인해 주세요.",
  "fieldErrors": [
    { "field": "password", "message": "size must be between 8 and 72" }
  ],
  "requestId": "req..."
}
```

HTTP	주요 code	프론트 처리
400	VALIDATION_ERROR	fieldErrors 를 각 input 필드 오류로 표시
401	UNAUTHORIZED , INVALID_CREDENTIALS , INVALID_REFRESH_TOKEN	토큰 갱신 또는 로그인 화면 이동

403	FORBIDDEN	다른 사용자 자원에 접근했음을 알림
404	*_NOT_FOUND	현재 화면의 캐시를 지우고 목록 재조회
409	EMAIL_ALREADY_EXISTS , FAVORITE_ALREADY_EXISTS	중복 상태를 안내
413/415	FILE_TOO_LARGE , UNSUPPORTED_IMAGE	이미지 용량/형식 안내
500	INTERNAL_ERROR	requestId 를 기록하고 일반 오류 문구 표시

2. 프론트 권장 연동 순서

1. 회원가입/로그인	2. 신체 프로필	3. 아바타 생성	4. 의류 2개 이상
5. TPO 분석	6. 날씨/추천	7. 착장 미리보기	8. 찜/피드백

- 1. 로그인: access/refresh token을 받고 API client에 bearer token을 설정한다.
- 2. 프로필: PUT /body-profile 후 POST /avatar/generate 를 호출한다.
- 3. 옷장: 서로 다른 카테고리의 의류를 2개 이상 업로드한다.
- 4. 문장→TPO: POST /tpo/parse 결과를 사용자가 확인·수정하게 한다.
- 5. 추천: 확정된 TPO 배열과 위치를 POST /recommendations 에 보낸다.
- 6. 착장: 빠른 결과는 /try-on/preview , AI 결과는 /try-on/render 후 GET polling을 사용한다.

선행 조건: 추천은 의류 2개 이상, 착장은 아바타가 필요하다. 화면에서 선행 조건을 먼저 검사하면 400 오류를 줄일 수 있다.

3. 전체 엔드포인트 요약

Method	Path	역할	응답
POST	/auth/signup	계정 생성	201
POST	/auth/login	access/refresh token 발급	200
POST	/auth/refresh	토큰 교체	200
POST	/auth/logout	refresh token 폐기	204
GET/PUT/DELETE	/users/me	내 계정 조회/수정/탈퇴	200/204
GET/PUT/DELETE	/body-profile	신체 프로필 조회/저장/삭제	200/204
POST/GET/PUT/DELETE	/avatar...	아바타 생성/조회/버전 생성/삭제	200/204
GET/POST	/wardrobe	의류 목록/이미지 업로드	200/201
GET/PUT/DELETE	/wardrobe/{itemId}	의류 상세/수정/삭제	200/204
POST	/wardrobe/{itemId}/reanalyze	의류 재분석 상태 갱신	200
POST	/tpo/parse	자연어 일정을 TPO 구조로 변환	200
GET	/weather/forecast	위치·시간 기준 날씨	200
POST	/recommendations	코디 추천 생성	201
GET	/recommendations/{id}	추천 결과 재조회	200
POST	/recommendations/{id}/regenerate	같은 조건으로 새 추천	201
POST	/try-on/preview	빠른 2D 착장	201
POST	/try-on/render	비동기 AI 착장	202
GET/DELETE	/try-on/{resultId}	착장 상태 조회/삭제	200/204
GET/POST	/favorites	찜 목록/추가	200/201
DELETE	/favorites/{favoriteId}	찜 해제	204
POST	/recommendations/{id}/feedback	추천 LIKE/DISLIKE 저장	201
POST	/outfits/feedback	사용자 선택 조합 채점	201

4. 인증 API

POST /api/v1/auth/signup **인증 불필요**

이유: 이메일/비밀번호 계정을 생성해 사용자별 옷장·추천 데이터의 소유자를 구분한다.

입력(JSON)	제약	반환
email*, password*, nickname*	email 형식, password 8~72자, nickname 1~60자	201 ApiResponse<UserResponse>

```
{ "email": "user@example.com", "password": "TestPass123!", "nickname": "코디핀" }
```

주요 오류: 409 EMAIL_ALREADY_EXISTS . 가입 후에도 token은 자동 발급되지 않으므로 login을 호출한다.

POST /api/v1/auth/login **인증 불필요**

이유: API 인증에 쓸 access token과 재발급에 쓸 refresh token을 발급한다.

입력(JSON)	반환	프론트 처리
email*, password*	200 ApiResponse<TokenResponse>	accessToken 을 Authorization 헤더에, refreshToken 을 보안 저장소에 보관

```
{
  "data": {
    "tokenType": "Bearer",
    "accessToken": "eyJ...",
    "accessExpiresIn": 3600,
    "refreshToken": "uuid.uuid",
    "refreshExpiresIn": 1209600
  },
  "meta": { "requestId": "req_..." }
}
```

주요 오류: 401 INVALID_CREDENTIALS .

POST /api/v1/auth/refresh **인증 불필요**

이유: access token 만료 시 로그인 유지. 사용한 refresh token은 폐기되고 새 refresh token이 발급된다.

입력(JSON)	반환	주의
refreshToken*	200 ApiResponse<TokenResponse>	응답의 access/refresh token 둘 다 기존 값을 교체

주요 오류: 401 INVALID_REFRESH_TOKEN . 이때는 로그아웃 처리한다.

POST**/api/v1/auth/logout**

인증 불필요

이유: refresh token을 서버에서 폐기해 다시 사용할 수 없게 한다.

입력(JSON)	반환	프론트 처리
refreshToken*	204 No Content	성공 여부와 무관하게 로컬 토큰을 모두 삭제

5. 사용자·신체 프로필·아바타 API

5.1 사용자

GET /api/v1/users/me **JWT**

이유: 앱 시작/새로고침 시 현재 로그인 사용자를 복원한다. 입력 없음. **반환:** 200 ApiResponse<UserResponse> .

PUT /api/v1/users/me **JWT**

이유: 닉네임, 퍼스널 컬러, 스타일 선호도를 추천용 프로필로 저장한다.

입력(JSON)	제약	반환
nickname* , personalColor , stylePreference	nickname 최대 60, personalColor 40, stylePreference 100자	200 ApiResponse<UserResponse>

DELETE /api/v1/users/me **JWT**

이유: 회원 데이터와 Firebase Storage의 users/{userId}/ 객체를 정리한다. **반환:** 204 . 성공 후 token/캐시를 모두 삭제하고 로그인 화면으로 이동한다.

5.2 신체 프로필

GET /api/v1/body-profile **JWT**

이유: 아바타 생성 전에 저장된 신체 프로필을 불러온다. 입력 없음. **반환:** 200 ApiResponse<BodyProfileResponse> . 없으면 404 BODY_PROFILE_NOT_FOUND .

PUT /api/v1/body-profile **JWT**

이유: 아바타 비율과 착장 설명에 사용할 신체 정보를 저장/갱신한다.

입력(JSON)	제약/권장값	반환
heightCm* , bodyType* , genderExpression* , consent*	heightCm 100~230; consent는 반드시 true ; bodyType 예: RECTANGLE , TRIANGLE , INVERTED_TRIANGLE ; genderExpression 예: NEUTRAL	200 ApiResponse<BodyProfileResponse>

```
{ "heightCm":170, "bodyType":"RECTANGLE", "genderExpression":"NEUTRAL", "consent":true }
```

DELETE /api/v1/body-profile **JWT**

이유: 신체 프로필과 연관 아바타를 제거한다. **반환:** 204 . 아바타를 참조하는 착장 결과도 DB cascade로 삭제될 수 있으므로 관련 프론트 캐시를 비운다.

5.3 아바타

POST /api/v1/avatar/generate **JWT**

이유: 신체 프로필로 기본 SVG 아바타를 생성해 착장의 바탕으로 사용한다. 입력 없음. **반환:** 200 ApiResponse<AvatarResponse> . 프로
필이 없으면 400 BODY_PROFILE_REQUIRED .

GET /api/v1/avatar **JWT**

이유: 현재 최신 아바타를 화면에 표시한다. 입력 없음. **반환:** 200 ApiResponse<AvatarResponse> . 없으면 404 AVATAR_NOT_FOUND .

PUT /api/v1/avatar **JWT**

이유: 템플릿/옵션을 반영한 새 버전의 아바타를 생성한다.

입력(JSON)	예시	반환
templateId* , options (자유 key/value)	{"templateId":"STANDARD_FRONT", "optio ns":{"skinTone":"MEDIUM"}}	200 ApiResponse<AvatarResponse> , version 1 증가

DELETE /api/v1/avatar **JWT**

이유: 아바타 DB 데이터와 저장 이미지를 삭제한다. **반환:** 204 . 성공 후 착장 관련 상태도 재조회한다.

6. 옷장 API

업로드 규칙: JPEG, PNG, WebP만 허용하며 최대 10MB다. 원본은 Firebase `wardrobe` , 현재 마스크 복사본은 `masks` 폴더에 저장된다.

GET

/api/v1/wardrobe

JWT

이유: 사용자 옷장 목록을 필터/페이징하여 조회한다.

Query	기본/제약	반환
category , season , color , page , size	page=0, size=20; 서버가 size를 1~100으로 보정	200 ApiResponse<WardrobeItemResponse[]>

총 개수/totalPages는 반환하지 않는다. 배열 길이가 size보다 작으면 마지막 페이지로 판단한다.

POST

/api/v1/wardrobe

JWT

이유: 의류 이미지와 속성을 저장해 추천/착장의 기본 데이터로 사용한다.

FormData key	필수/권장값	반환
image* (File), name* , category* , color , season , warmth , formality , fit , length	category: TOP/BOTTOM/OUTER/SHOES/OTHE R; warmth: 1~5; season: ALL/SPRING/SUMMER/FALL/WINTER ; formality: CASUAL/SMART_CASUAL/FORMAL/S PORT	201 ApiResponse<WardrobeItemResponse>

선택값을 비우면 color=미분류, season=ALL, warmth=3, formality=CASUAL, fit/length=REGULAR로 저장된다.

GET

/api/v1/wardrobe/{itemId}

JWT

이유: 의류 상세/수정 화면을 위해 한 건을 조회한다. **입력:** path `itemId*` . **반환:** 200 ApiResponse<WardrobeItemResponse> .

PUT

/api/v1/wardrobe/{itemId}

JWT

이유: 업로드 후 사용자가 분석 속성을 확인/수정한다.

입력(JSON)	필수/제약	반환
name* , category* , color , season , warmth , formality , fit , length , description	warmth 1~5. PUT이므로 화면의 전체 필드를 보내는 것을 권장	200 ApiResponse<WardrobeItemResponse> , analysisStatus= CONFIRMED

DELETE

/api/v1/wardrobe/{itemId}

JWT

이유: DB 의류 데이터와 Firebase의 원본/마스크 객체를 함께 제거한다. **반환:** 204 . 목록에서 즉시 제거한다.

POST `/api/v1/wardrobe/{itemId}/reanalyze` **JWT**

이유: 의류 설명/분석 상태를 재생성하는 UI 흐름을 제공한다. 현재 서버는 기본 설명을 재생성하고 `NEEDS_CONFIRMATION` 을 반환한다.

반환: `200 ApiResponse<WardrobeItemResponse>` .

7. TPO·날씨 API

POST /api/v1/tpo/parse **JWT**

이유: “오전에 학교, 저녁에 데이트”와 같은 문장을 추천 API가 사용할 구조화된 일정으로 변환한다.

입력(JSON)	형식	반환
text* , baseDate , timezone	baseDate YYYY-MM-DD ; timezone IANA 예: Asia/Seoul . 생략 시 오늘/서울	200 ApiResponse<TpoParseResponse>

```
{ "text": "오전에 학교, 저녁에 데이트", "baseDate": "2026-08-20", "timezone": "Asia/Seoul" }
```

needsUserConfirmation 은 현재 항상 true다. segments를 바로 추천에 넘기기 전 사용자가 시간/장소/활동/격식을 확인하게 한다.

GET /api/v1/weather/forecast **JWT**

이유: 위치/시간별 기온·강수·바람을 보여주고 추천 조합에 사용한다.

Query	형식	반환
latitude* , longitude* , at	at=ISO 8601 offset datetime; 생략 시 현재 한국 시간	200 ApiResponse<WeatherResponse>

fallback=true 또는 source="FALLBACK" 이면 기상청 실시간값이 아닌 계절 기본값이다. UI에 “예상 날씨” 안내를 표시한다.

8. 코디 추천 API

POST /api/v1/recommendations **JWT**

이유: TPO, 위치/날씨, 희망 분위기와 보유 의류를 종합해 1~3개 코디를 생성한다.

입력(JSON)	제약	반환
tpoSegments* (1개 이상), location*, desiredMood, maxOutfits	latitude -90~90, longitude -180~180, maxOutfits 1~3(기본 3)	201 ApiResponse<RecommendationResponse>

```
{
  "tpoSegments": [{
    "startAt": "2026-08-20T19:00:00+09:00",
    "locationText": "해운대", "activity": "데이트", "formality": "SMART_CASUAL"
  }],
  "location": { "latitude": 35.1587, "longitude": 129.1604 },
  "desiredMood": "깔끔한 여름밤", "maxOutfits": 3
}
```

선행 조건: 의류 2개 이상. 부족하면 400 INSUFFICIENT_WARDROBE . 응답의 outfits[].id 가 찜/착장에 사용할 outfitId 다.

GET /api/v1/recommendations/{id} **JWT**

이유: 추천 상세 화면/새로고침에서 저장된 결과를 복원한다. **입력:** path id* . **반환:** 200 ApiResponse<RecommendationResponse> . 없거나 내 것이 아니면 404 RECOMMENDATION_NOT_FOUND .

POST /api/v1/recommendations/{id}/regenerate **JWT**

이유: 기존 조건을 다시 입력하지 않고 새 추천 레코드를 생성한다. request body 없음. **반환:** 201 ApiResponse<RecommendationResponse> . 반환된 새 id 로 화면 상태를 교체한다.

9. 가상 착장 API

POST /api/v1/try-on/preview **JWT**

이유: AI 대기 없이 아바타와 의류 마스크를 합성한 SVG 미리보기를 즉시 생성한다.

입력(JSON)	선행 조건	반환
outfitId*	내 추천 outfit + 아바타	201 ApiResponse<TryOnResponse> , status= COMPLETED , renderType= PREVIEW , previewUrl 즉시 제공

POST /api/v1/try-on/render **JWT** **비동기**

이유: 의류 원본을 OpenAI 이미지 편집에 전달해 고품질 착장을 시도한다. API key/호출이 없거나 실패하면 2D SVG로 자동 대체한다.

입력(JSON)	즉시 반환	최종 상태
outfitId*	202 ApiResponse<TryOnResponse> , status= PROCESSING ; previewUrl은 null 이며 JSON에서 생략될 수 있음	COMPLETED (AI), COMPLETED_FALLBACK (2D), FAILED

Polling: 응답의 id 를 받은 후 1.5~2초 간격으로 GET /try-on/{id} 를 호출한다. 60초 정도의 클라이언트 timeout을 두고 화면 이탈 시 polling을 취소한다.

GET /api/v1/try-on/{resultId} **JWT**

이유: 비동기 렌더 상태/결과 이미지를 조회한다. **반환:** 200 ApiResponse<TryOnResponse> . previewUrl 이 생기고 status가 PROCESSING이 아니면 polling을 종료한다.

DELETE /api/v1/try-on/{resultId} **JWT**

이유: 착장 결과 DB 행과 Firebase 결과 이미지를 삭제한다. **반환:** 204 .

10. 찜·피드백 API

GET /api/v1/favorites **JWT**

이유: 사용자가 저장한 코드 스냅샷과 미리보기 URL을 목록으로 조회한다. 입력 없음. **반환:** 200 ApiResponse<FavoriteResponse[]> .

POST /api/v1/favorites **JWT**

이유: 추천 outfit과 현재 착장 이미지를 다시 볼 수 있도록 저장한다.

입력(JSON)	설명	반환
outfitId* , previewUrl	previewUrl은 try-on 결과가 있을 때 전달; 없으면 생략 가능	201 ApiResponse<FavoriteResponse>

같은 outfit을 두 번 찜하면 409 FAVORITE_ALREADY_EXISTS .

DELETE /api/v1/favorites/{favoriteId} **JWT**

이유: 찜 목록에서 저장 항목을 해제한다. 주의: path는 outfitId 가 아니라 FavoriteResponse.id 다. **반환:** 204 .

POST /api/v1/recommendations/{id}/feedback **JWT**

이유: 특정 추천에 대한 사용자 선호를 저장해 후속 품질 개선에 사용한다.

입력(JSON)	제약	반환
rating* , reasonCode , comment , outfitId	rating= LIKE 또는 DISLIKE ; comment 최대 500자; outfitId는 해당 추천의 특정 코드를 평가할 때	201 , body 없음

POST /api/v1/outfits/feedback **JWT**

이유: 추천이 아닌 사용자 직접 선택 의류 조합을 색상/핏/격식/날씨로 채점하고 조언한다.

입력(JSON)	필수 로직	반환
wardrobeItemIds , tpoSegments , location , weather , desiredMood	DTO상 optional이지만 서비스에서 wardrobeItemIds 2개 이상 필수	201 ApiResponse<OutfitFeedbackResponse>

아바타가 있으면 preview 가 함께 반환되고, 없으면 null이며 JSON에서 생략될 수 있다. 의류가 부족하면 400 OUTFIT_ITEMS_REQUIRED .

11. 공통 DTO 필드 사전

* 는 필수. nullable 필드는 Spring의 non-null JSON 설정으로 값이 null이면 응답에서 생략될 수 있다.

11.1 프로필·옷장

DTO	필드
UserResponse	id, email, nickname, personalColor?, stylePreference?, createdAt
BodyProfileResponse	userId, heightCm, bodyType, genderExpression, consentAt, updatedAt
AvatarResponse	id, templateId, parameters (object), imageUrl, version, createdAt
WardrobeItemResponse	id, name, category, color?, season?, warmth?, formality?, fit?, length?, description?, imageUrl, maskUrl?, anchor, analysisStatus, createdAt
anchor	x, y, width, height, layer . 400×700 SVG 착장 캔버스에서의 배치 기준값.

11.2 TPO·날씨·추천

DTO	필드
TpoSegment	startAt, locationText, activity, formality
TpoParseResponse	segments: TpoSegment[], needsUserConfirmation
Location	latitude* (-90~90), longitude* (-180~180)
WeatherResponse	latitude, longitude, forecastAt, temperatureC, feelsLikeC, humidityPercent, precipitationMm, windSpeedMs, dailyRangeC, condition, source, fallback
RecommendationResponse	id, tpoSegments, weather, status, outfits: OutfitResponse[], createdAt
OutfitResponse	id, rank, score, wardrobeItemIds, reason, weatherTip, fitTip, alternativeItemIds

11.3 착장·찜·평가

DTO	필드
TryOnResponse	id, avatarId, outfitId, previewUrl?, renderType, status, expiresAt (생성+30일)
FavoriteResponse	id, outfitId, outfit: OutfitResponse, previewUrl?, createdAt
OutfitFeedbackResponse	id, totalScore, colorScore, fitScore, formalityScore, weatherScore, comment, suggestions, preview?

11.4 권장 문자열 값

필드	권장값	비고
category	TOP, BOTTOM, OUTER, SHOES, OTHER	서버가 대문자로 정규화
season	ALL, SPRING, SUMMER, FALL, WINTER	서버 enum validation은 없으나 프론트에서 고정 선택지 권장
formality	CASUAL, SMART_CASUAL, FORMAL, SPORT	TPO formality와 같은 값 체계 사용
warmth	1~5	1=엄청 얇음/더운 날, 5=두꺼움/추운 날
fit	SLIM, REGULAR, LOOSE, OVERSIZED	고정 선택지 권장

length	CROPPED, SHORT, REGULAR, LONG	고정 선택지 권장
analysisStatus	NEEDS_CONFIRMATION, CONFIRMED	프론트는 읽기 전용
Try0n.status	PROCESSING, COMPLETED, COMPLETED_FALLBACK, FAILED	PROCESSING일 때만 polling

12. TypeScript 연동 예시

12.1 공통 client

```
const API_BASE = import.meta.env.VITE_API_BASE_URL;

export async function apiFetch<T>(path: string, init: RequestInit = {}) {
  const accessToken = authStore.getState().accessToken;
  const response = await fetch(`${API_BASE}${path}`, {
    ...init,
    headers: {
      ...(init.body instanceof FormData ? {} : { "Content-Type": "application/json" }),
      ...(accessToken ? { Authorization: `Bearer ${accessToken}` } : {}),
      ...init.headers,
    },
  });

  if (response.status === 204) return undefined as T;
  const body = await response.json();
  if (!response.ok) throw body; // code, message, fieldErrors, requestId
  return body as T;
}
```

12.2 의류 업로드

```
const form = new FormData();
form.append("image", file);
form.append("name", values.name);
form.append("category", values.category);
if (values.color) form.append("color", values.color);
if (values.season) form.append("season", values.season);
if (values.warmth) form.append("warmth", String(values.warmth));

const result = await apiFetch<ApiResponse<WardrobeItem>>(<
  "/api/v1/wardrobe",
  { method: "POST", body: form }
>);
```

12.3 AI 착장 polling

```
const started = await apiFetch<ApiResponse<TryOnResult>>(<
  "/api/v1/try-on/render",
  { method: "POST", body: JSON.stringify({ outfitId }) }
>);

let result = started.data;
for (let i = 0; i < 30 && result.status === "PROCESSING"; i++) {
  await new Promise(resolve => setTimeout(resolve, 2000));
  result = (await apiFetch<ApiResponse<TryOnResult>>(<
    `/api/v1/try-on/${result.id}`
  >)).data;
}

if (["COMPLETED", "COMPLETED_FALLBACK"].includes(result.status)) {
  showImage(result.previewUrl!);
} else {
  showRenderError();
}
```

12.4 최소 공통 타입

```
export interface ApiResponse<T> {
  data: T;
  meta: { requestId: string };
}

export interface ApiError {
  code: string;
  message: string;
  fieldErrors: Array<{ field: string; message: string }>;
  requestId?: string;
}

export interface TpoSegment {
  startAt: string;
  locationText: string;
  activity: string;
  formality: "CASUAL" | "SMART_CASUAL" | "FORMAL" | "SPORT";
}

export interface TryOnResult {
  id: string;
  avatarId: string;
  outfitId: string;
  previewUrl?: string;
  renderType: "PREVIEW" | "AI" | "AI_FALLBACK";
  status: "PROCESSING" | "COMPLETED" | "COMPLETED_FALLBACK" | "FAILED";
  expiresAt: string;
}
```

13. 오류 처리·QA 체크리스트

13.1 프론트 오류 처리 우선순위

- 1. **HTTP status**로 큰 흐름(401 토큰, 403 권한, 404 삭제됨, 409 중복)을 결정한다.
- 2. **code**로 사용자 안내 문구/화면 분기를 결정한다.
- 3. **fieldErrors**를 form input 아래에 표시한다.
- 4. **requestId**를 클라이언트 로그/오류 제보에 함께 남긴다.

13.2 기능별 QA

시나리오	기대 결과
로그인→새로고침	GET /users/me 로 사용자 복원. access 만료 시 refresh 후 1회 재시도.
중복 이메일	409 안내, 가입 form 유지.
10MB 초과/지원 안 함	413 또는 415 안내, 업로드 재시도 가능.
추천 전 의류 부족	INSUFFICIENT_WARDROBE 안내 후 옷장 등록으로 이동.
착장 전 아바타 없음	AVATAR_REQUIRED 안내 후 프로필/아바타 화면으로 이동.
AI 키 없음/실패	COMPLETED_FALLBACK 과 2D previewUrl을 성공 결과로 표시.
회원 탈퇴	204 후 token, user, wardrobe, recommendation, favorite, try-on 캐시 전부 제거.

프론트 완료 기준: 인증 갱신, 204 빈 body, FormData 업로드, nullable 필드, AI polling, fallback 상태, 삭제 후 캐시 정리까지 모두 테스트한다.